

**THE UNIVERSITY
OF QUEENSLAND**
A U S T R A L I A

Ranking Without Reliability Evaluating Retrieval-Stage Data Poisoning Attacks Against LLM-Reranked RAG Pipelines

COMS4507 - Advanced Topics in Security and Privacy
Group Project Report

by

Team F

Li Haoyu (48661832)

Li Xiang (49729775)

Liu Dongchen (49133149)

Liu Jinming (49061091)

School of Electrical Engineering and Computer Science
The University of Queensland.

May 2026

Use of AI Statement

I acknowledge the use of generative AI tools in completing this assignment. Details of which tools were used and how they were used are provided in the table below, along with appropriate in-text and full references. I take responsibility for critically evaluating and integrating the AI-generated content, and ensuring it adheres to academic integrity standards.

AI Model and date(s)	Table A1. Artificial Intelligence (AI) tool usage (tick all that apply)											
	Language Translation	Grammar / style / spelling	Topic exploration	Research question	Literature review	Analyze data	Generate data	Generate text	Content Creation visual (formatting)	Content Creation (e.g. code)	Feedback	Other (provide details)
<i>ChatGPT</i> May 2026	✓	✓	✓	✓	✓			✓			✓	
<i>Claude</i> May 2026	✓	✓	✓	✓	✓			✓			✓	
<i>Doubao</i> May 2026	✓	✓	✓	✓	✓			✓			✓	
<i>Google Gemini</i> May 2026	✓	✓	✓	✓	✓			✓			✓	

Generative AI tools were used in a limited and supportive capacity during the preparation of this project report. ChatGPT, Claude, Doubao, and Google Gemini were used in May 2026 to assist with language translation, grammar, style and spelling improvement, topic exploration, refinement of the research question, literature review support, draft text generation, and feedback on the structure and wording of the report. These tools helped improve the clarity, organisation, and academic expression of the report, particularly when explaining the RAG poisoning threat model, retrieval-stage evaluation design, experimental findings, and discussion of implications.

The project data, including the corpus, poison sets, experiment results, ASR values, tables, and figures, were derived from the submitted project repository and verified by the authors. All AI-generated suggestions were reviewed, edited, and validated by the authors before inclusion. The authors take full responsibility for the research design, interpretation of results, discussion, limitations, references, and final written content. All information and sources were checked to ensure accuracy and compliance with academic integrity requirements.

Abstract

Retrieval-Augmented Generation (RAG) systems improve large language model (LLM) applications by grounding answers in an external knowledge base. This design improves factuality and updateability, but it also shifts part of the system's trust boundary from the model to the corpus. If an attacker can insert even a small number of crafted documents into the knowledge base, those documents may be retrieved and presented to the downstream LLM as apparently relevant context.

This report presents a systematic evaluation of data poisoning attacks against the retrieval and reranking stages of a RAG pipeline. The evaluation deliberately focuses on retrieval-layer rank shift rather than final-answer manipulation: an attack is considered successful when at least one poison document enters the top-k retrieval or reranking results. The main experiment covers 30 queries, five poisoning strategies, and four LLM rerankers, producing 600 trials. Three principal findings emerge. First, Authority Spoof is the most universal vulnerability, reaching 97% Attack Success Rate (ASR) across all four rerankers. Second, Contradiction attacks reveal the largest model-level robustness gap, ranging from 77% ASR for Claude Sonnet 4.5 to 90% ASR for Llama 3.3 70B. Third, Keyword Stuffing is the only attack type substantially reduced by reranking, dropping from 73% ASR at dense retrieval to 59% after LLM reranking.

These results suggest that LLM-based reranking can filter some surface-level manipulation, but should not be treated as a sufficient defence against semantically plausible or authority-style poisoning. The project releases the corpus, poison sets, 600-row result dataset, plots, and a Streamlit demonstration interface to support reproducibility.

Contents

Use of AI Statement.....	i
Abstract.....	iii
1. Introduction.....	1
1.1 Research Contributions	1
1.2 Scope and Delimitation	2
2. Related Work.....	3
2.1 RAG and the Corpus Trust Boundary	3
2.2 Knowledge Corruption Attacks: PoisonedRAG.....	3
2.3 Denial-of-Service via Jamming: Machine Against the RAG	3
2.4 Prompt Injection Defence: StruQ.....	3
2.5 Positioning of This Study	4
3. Threat Model and Problem Setup	5
3.1 Attacker Capability and Knowledge	5
3.2 Attack Scale.....	5
3.3 Attack Objective and Formal Success Condition.....	5
3.4 Evaluation Metrics	5
4. Methodology	6
4.1 System Pipeline.....	6
4.2 Corpus Construction.....	6
4.3 Test Query Set	6
4.4 Poison Document Generation Strategies.....	7
4.5 Dense Retrieval Implementation.....	7
4.6 LLM Reranking Configuration	8
4.7 Generator.....	8
4.8 Batch Experiment Procedure.....	8
5. Results.....	9
5.1 Overall Attack Behaviour.....	9
5.2 Complete k2 ASR Matrix.....	9
5.3 F1: Authority Spoof Is a Universal Vulnerability	10
5.4 F2: Contradiction Shows the Largest Cross-Model Robustness Gap	10
5.5 F3: Keyword Stuffing Is the Only Attack Clearly Weakened by Reranking	11
5.6 O1: Reranker Output Completeness Differs Across LLMs	11
5.7 O2: Gemini Does Not Simply Pass Through Dense Order	11
6. Discussion	12
6.1 Authority Cues Are a Structural Weakness for LLM Reranking.....	12

6.2 Model Choice Matters, but Is Not a Defence by Itself	12
6.3 Reranking Helps Mainly Against Surface-Level Manipulation.....	12
6.4 Contradiction Results Challenge Intuitive Expectations.....	13
6.5 Telemetry Is Part of RAG Security Engineering.....	13
6.6 Implications for Production RAG Deployments.....	13
7. Limitations	14
7.1 Limited Corpus Scale	14
7.2 Single Fixed Generator	14
7.3 GPT-4o-mini Output Completeness and Sensitivity Analysis	14
7.4 OpenRouter Capacity Variability	14
7.5 Research Demonstration Scope.....	14
8. Conclusion and Future Work	15
References.....	16

1. Introduction

Retrieval-Augmented Generation has become a common architecture for knowledge-intensive LLM applications. Rather than relying only on information stored in model parameters, a RAG system retrieves documents from an external knowledge base and supplies them as context to the generator [1]. This design can improve freshness, domain specificity, and traceability, especially in applications such as customer support, legal research, enterprise search, and university information services. However, it also introduces a security dependency that is easy to underestimate: the quality and trustworthiness of the generated answer now depend on the integrity of the corpus.

The corpus is often more open and dynamic than the model itself. In realistic deployments, knowledge bases may ingest scraped web pages, public reviews, third-party documents, user uploads, or semi-curated content. These sources can include untrusted or weakly verified material. An attacker who can inject a small number of documents into this corpus may not need to control the LLM or the retriever. It may be sufficient for the malicious documents to be retrieved for a target query and inserted into the context window. Once that happens, the downstream LLM is exposed to attacker-controlled information.

This project studies that upstream attack surface. The core research question is: can a small number of poison documents alter the top-k ranking of a RAG pipeline before generation occurs? The framing is intentionally narrower than measuring whether the final LLM answer is false. If a malicious document reaches the retrieval context, the system has already lost an important security property: context selection has become vulnerable to attacker influence. For this reason, the project defines attack success as rank change, specifically whether at least one poison document enters top-k1 dense retrieval or top-k2 LLM-reranked results.

This focus separates the project from conventional prompt injection work. Prompt injection typically studies whether a model follows malicious instructions contained in retrieved text. This report instead asks whether malicious documents can reach the context window in the first place. It also separates the project from final-answer poisoning studies, which measure whether the generator produces an attacker-chosen response. Retrieval-stage evaluation is valuable because it isolates a measurable failure point before generation and allows reranker models to be compared without multiplying the experiment by generator choices.

1.1 Research Contributions

1. A retrieval-stage poisoning evaluation pipeline that compares clean and poisoned rankings at both k1 and k2.
2. A systematic comparison of five poisoning strategies: Keyword Stuffing, Structured Format, Semantic Mimicry, Authority Spoof, and Contradiction.
3. A multi-reranker robustness comparison across anthropic/claude-sonnet-4.5, openai/gpt-4o-mini, google/gemini-2.5-flash-lite, and meta-llama/llama-3.3-70b-instruct.
4. Empirical evidence that Authority Spoof is a near-universal vulnerability, reaching 97% ASR across all four evaluated rerankers.
5. A reproducible research demo containing corpus files, poison sets, experiment CSV, plots, source code, and a Streamlit visualisation interface.

1.2 Scope and Delimitation

The scope of this report is the retrieval and reranking stages of a RAG pipeline. The generator is included in the demonstration interface to show the end-to-end system, but generated answers are not used as quantitative evidence. The main metric is rank shift, not output correctness. This delimitation is central to the project: it keeps the evaluation focused on the earliest point where attacker-controlled content can enter the context window.

2. Related Work

2.1 RAG and the Corpus Trust Boundary

RAG combines parametric language models with non-parametric external memory [1]. In the original formulation, retrieved documents provide evidence that can improve knowledge-intensive generation and help update the system without retraining. From a security perspective, the same architecture creates a corpus trust boundary. If the retriever selects untrusted content, the generator may treat it as part of the evidential context. This project focuses precisely on that boundary: the point where retrieval decides which documents become visible to the LLM.

2.2 Knowledge Corruption Attacks: PoisonedRAG

PoisonedRAG is the closest prior work to this project [4]. Zou et al. study knowledge corruption attacks in which an attacker injects malicious texts into a RAG knowledge database to induce the downstream LLM to produce attacker-chosen answers. Their method recognises that a successful poison must satisfy both a retrieval condition and a generation condition. This is highly relevant to the present project because both works treat the knowledge base as a practical attack surface.

The difference is the success criterion. PoisonedRAG evaluates whether the final generated answer is manipulated. This project evaluates an earlier stage: whether poison documents enter top-k retrieval or reranking results. This distinction matters because a rank shift can be detected and defended against before generation occurs. It also allows the experiment to compare several LLM rerankers while holding the generator fixed.

2.3 Denial-of-Service via Jamming: Machine Against the RAG

Machine Against the RAG studies jamming attacks against RAG systems [2]. In that setting, the attacker injects a blocker document designed to cause the system to refuse to answer a target query. The threat model is close to this project: the attacker can add documents to the database but cannot control the retriever or LLM. However, the attack objective is different. Jamming aims for denial-of-service or refusal; this project aims to measure influence over retrieval rankings.

This comparison is useful because it shows that knowledge-base injection can create multiple classes of RAG failure. A malicious document may cause refusal, induce a false answer, or simply enter the top-k context. This report focuses on the third case because it is a necessary influence channel for many downstream failures.

2.4 Prompt Injection Defence: StruQ

StruQ proposes structured queries as a defence against prompt injection [3]. It separates control instructions from data so that an LLM can be trained to follow instructions only from the prompt region and ignore instructions embedded in the data region. StruQ is relevant because RAG systems often pass retrieved documents into a prompt, and those documents may contain untrusted text.

However, StruQ operates at the generation layer. It assumes that untrusted documents may already be present in the context window and tries to prevent the model from following malicious instructions within them. This project is upstream of that defence boundary. It evaluates whether poisoned documents reach the context window in the first place. The two directions are therefore complementary rather than competing: prompt-injection defences reduce exploitation after retrieval, while retrieval-stage poisoning evaluation measures whether the context selection process is already compromised.

2.5 Positioning of This Study

Work	Main Layer	Success Criterion	Difference from This Study
RAG [1]	Retrieval + generation architecture	Improved knowledge-intensive generation	Provides the architectural basis rather than an attack evaluation.
Machine Against the RAG [2]	Knowledge-base injection for jamming	System refuses to answer	Targets denial-of-service; this study targets rank shift.
StruQ [3]	Prompt injection defence	LLM ignores malicious instructions	Defends after retrieval; this study evaluates whether poison reaches top-k.
PoisonedRAG [4]	Knowledge corruption attack	Final answer matches attacker target	Evaluates generation outcome; this study isolates retrieval and reranking.
This study	Retrieval and LLM reranking	Poison document enters top-k	Compares five attacks across four LLM rerankers.

Table 1. Positioning of this study relative to related work.

3. Threat Model and Problem Setup

3.1 Attacker Capability and Knowledge

The attacker can inject a small set of poison documents into the knowledge base. The attacker cannot remove, read, or modify existing clean documents; cannot change the embedding model; cannot alter the FAISS index implementation; cannot modify the LLM reranker; and cannot modify the generator. This models a weak but realistic attacker in a partially trusted corpus, where the system may ingest public or semi-public content.

The attacker is assumed to know the target query text and can craft poison documents around that query. The attacker does not require model weights, internal embeddings, reranker hidden states, or generator outputs. The experiment is therefore not a white-box optimisation attack against a specific retriever. It evaluates whether simple and interpretable poison strategies can still change rankings under a controlled RAG pipeline.

3.2 Attack Scale

The static corpus contains 5,290 documents: 290 Brisbane topic documents and 5,000 MS MARCO background documents. Each poison set contains between 22 and 30 documents, representing less than 1% of the full corpus. This small attack scale is important because it prevents the evaluation from becoming a trivial corpus-flooding experiment. A successful attack under this setting indicates that a small number of targeted documents can influence top-k retrieval.

3.3 Attack Objective and Formal Success Condition

Let S denote the clean knowledge base, P_x denote one poison set, and q denote a target query. The poisoned knowledge base is $S \cup P_x$. The attack objective is to cause at least one poison document p in P_x to appear in the ranked result list for q at either k_1 or k_2 . The key point is that success is defined at the ranking layer. The attacker does not need the downstream LLM to produce any particular answer.

An attack succeeds at k_1 if at least one poison document enters the dense retriever top- k_1 results. It succeeds at k_2 if at least one poison document remains in the LLM reranker top- k_2 results. Operationally, this corresponds to the experimental columns `k1_attack_success` and `k2_attack_success`. This rank-change definition is the central evaluation boundary of the project.

3.4 Evaluation Metrics

Metric	Meaning
<code>poison_in_topk</code>	Binary indicator showing whether a poison document appears in the evaluated top-k list.
<code>poison_rank</code>	Best rank achieved by any poison document; lower rank indicates a stronger attack.
<code>n_poison_in_topk</code>	Number of poison documents appearing in the evaluated top-k list.
<code>displaced_docs</code>	Clean document identifiers pushed out of top-k by poison injection.
<code>score_gap</code>	Best poison score minus clean top-1 score.
<code>ASR</code>	Attack Success Rate: proportion of query-level trials where <code>attack_success</code> is true.
<code>reranker_padded_clean</code> / <code>poisoned</code>	Telemetry showing whether the LLM reranker emitted an incomplete listwise ranking that required parser backfilling.

Table 2. Evaluation metrics recorded by the experimental pipeline.

4. Methodology

4.1 System Pipeline

The implemented pipeline follows seven stages: Query -> Embedding -> FAISS top-k1 -> LLM Reranker -> top-k2 -> LLM Generator -> Answer. Each experiment runs the same query twice. The clean path uses only the static knowledge base S . The poisoned path injects one poison set P_x at runtime, runs the same retrieval and reranking procedure, and removes the poison documents before the next trial. Dense retrieval is evaluated at $TOP_K_1 = 10$, and LLM reranking is evaluated at $TOP_K_2 = 5$.

The code structure reflects this design. `src/pipeline.py` coordinates clean and poisoned runs. `src/retriever.py` implements FAISS retrieval and runtime poison injection/removal. `src/reranker.py` implements the listwise LLM reranker. `src/evaluator.py` compares clean and poisoned rankings. `scripts/run_experiment.py` executes the full batch experiment and writes the final CSV.

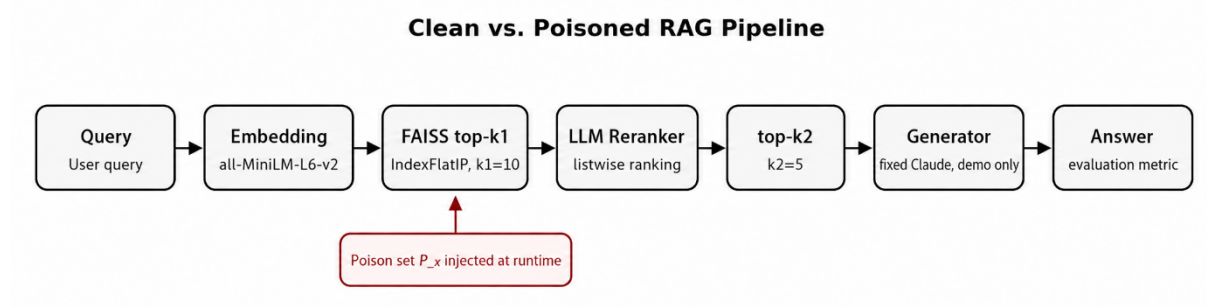


Figure 1. Clean-versus-poisoned RAG pipeline used in the experiment.

4.2 Corpus Construction

The knowledge base is designed as a two-part corpus. The Brisbane topic corpus contains 290 documents covering tourism, restaurants, transport, culture and food, and university-related information. It provides the domain-specific context for the test queries. The background corpus contains 5,000 MS MARCO passages sampled with seed=42 [6]. These passages act as general background noise and make the retrieval task less toy-like than a small topic-only corpus.

Corpus Component	Document Count	Purpose
Brisbane topic corpus	290	Domain-specific documents for Brisbane tourism, restaurants, transport, culture/food, and university queries.
MS MARCO background corpus	5,000	General background noise to make retrieval more realistic.
Total static corpus	5,290	Knowledge base S used before runtime poison injection.

Table 3. Static corpus composition.

4.3 Test Query Set

The experiment uses 30 manually curated queries across five categories aligned with the Brisbane corpus. Each query is paired with a `poison_target` and a `target_type` in

data/query_targets.yaml. The target types are 18 fictional entities, 8 false facts, and 4 misleading recommendations. This labelling allows poison documents to be generated toward a clear objective while keeping the evaluation comparable across attack types.

The query set is a controlled benchmark rather than a comprehensive survey of Brisbane travel questions. Its purpose is to test whether different poison styles can alter rankings under comparable conditions.

4.4 Poison Document Generation Strategies

The project evaluates five poisoning strategies. They are intentionally interpretable rather than heavily optimised, because the goal is to characterise how common document-design patterns affect retrieval and LLM reranking.

Attack	Layer	Generation	Coverage	Key Mechanism
Keyword Stuffing	Surface	Template-based	22 docs	Repeats query terms and poison_target phrases in shuffled chunks with limited semantic coherence.
Structured Format	Surface	openai/gpt-4o, temp 0.5	22 docs	Uses headings, bullet points, FAQ-style layout, and official-looking structure without strong authority claims.
Semantic Mimicry	Framing	openai/gpt-4o, temp 0.7	30 docs	Produces fluent content that resembles legitimate corpus style and appears semantically relevant.
Authority Spoof	Framing	openai/gpt-4o, temp 0.5	30 docs	Adds fabricated institutional references, DOI-like strings, rankings, or expert-sounding attribution.
Contradiction	Reasoning	openai/gpt-4o, temp 0.5	25 docs	Contradicts verifiable facts retrieved from clean documents while promoting the poison target.

Table 4. Poison document generation strategies.

LLM-generated poison documents use openai/gpt-4o as a fixed generation model. The target length is 100-200 words, with a validator tolerance up to 220 words. The final poison sets contain 129 documents: 30 Authority Spoof, 25 Contradiction, 22 Keyword Stuffing, 30 Semantic Mimicry, and 22 Structured Format documents. Keyword Stuffing and Structured Format exclude false-fact targets, while Contradiction excludes queries where the clean corpus lacks a fact that can be contradicted.

4.5 Dense Retrieval Implementation

Dense retrieval uses sentence-transformers/all-MiniLM-L6-v2, a 384-dimensional sentence embedding model suitable for semantic search [7], [8]. The FAISS index uses IndexFlatIP [5]. Both corpus vectors and query vectors are L2-normalised, so inner product search corresponds to cosine similarity. This choice keeps retrieval deterministic, efficient, and easy to audit.

Runtime poison injection is implemented by embedding the selected poison documents, normalising them, adding them to the existing FAISS index, and marking them as poison. After the poisoned run, the index is rebuilt from retained clean documents so the next experiment starts from a clean state. This design allows clean and poisoned rankings to be compared under the same pipeline without contamination across trials.

4.6 LLM Reranking Configuration

The top-k1 = 10 candidates are passed to an LLM reranker using a listwise prompt. The prompt presents numbered document snippets and asks the LLM to return only a comma-separated ordering of document numbers by relevance. Temperature is fixed at 0.0 for all reranker calls. Listwise reranking was selected because it allows candidate documents to be judged together rather than scored independently.

Short Name	Full Model ID	Role
Claude	anthropic/claude-sonnet-4.5	Reranker comparison model
GPT-4o-mini	openai/gpt-4o-mini	Reranker comparison model
Gemini	google/gemini-2.5-flash-lite	Reranker comparison model
Llama	meta-llama/llama-3.3-70b-instruct	Reranker comparison model

Table 5. LLM reranker models used in the final experiment.

All rerankers are accessed through OpenRouter. During the v1 experiment, Gemini was switched from google/gemini-2.0-flash-001 to google/gemini-2.5-flash-lite because the former was close to deprecation on OpenRouter and repeatedly hit rate-limit capacity. All final Gemini rows in `expr_v1_final.csv` use google/gemini-2.5-flash-lite. This disclosure is retained to make the experimental record transparent.

If the LLM returns fewer than the expected 10 valid indices, the parser appends the missing indices in their original dense-retrieval order. The number of parser-filled positions is recorded in `reranker_padded_clean` and `reranker_padded_poisoned` so that malformed reranker outputs can be audited.

4.7 Generator

A single fixed generator, anthropic/claude-sonnet-4.5, is used in the demonstration interface to produce natural-language answers from the top-k2 context. The generator is not a research variable. Fixing the generator avoids a reranker-by-generator combinatorial explosion and keeps the quantitative analysis focused on retrieval and reranking.

4.8 Batch Experiment Procedure

The main experiment executes every combination of 30 queries, five poison sets, and four reranker LLMs, producing 600 rows. For each row, the procedure is:

1. Run dense retrieval on the clean corpus and record top-k1.
2. Run the selected LLM reranker on the clean top-k1 and record top-k2.
3. Inject the selected poison set at runtime.
4. Run dense retrieval on the poisoned corpus and record top-k1.
5. Run the same LLM reranker on the poisoned top-k1 and record top-k2.
6. Compare clean and poisoned rankings at k1 and k2.
7. Record attack success, poison rank, displaced documents, score gap, padding telemetry, elapsed time, and errors.
8. Remove the poison set before the next experiment.

The final results are stored in `data/results/expr_v1_final.csv`, with plots stored as `expr_v1_final.png` and `expr_v1_final_by_llm.png`.

5. Results

5.1 Overall Attack Behaviour

Across the 600 trials, semantic and authority-style attacks are substantially stronger than shallow lexical manipulation. Authority Spoof achieves the highest and most stable k2 ASR. Semantic Mimicry is also highly effective, with k2 ASR above 93% for all rerankers. Structured Format and Contradiction remain effective but show more model-level variation. Keyword Stuffing is the weakest attack after reranking.

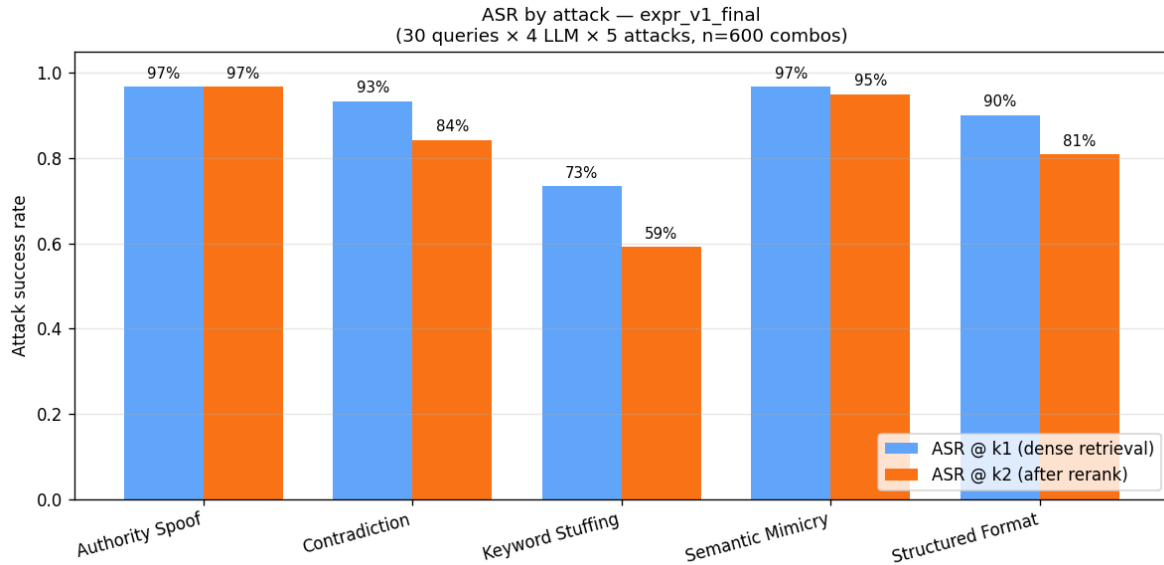


Figure 2. Average ASR of five attacks across dense retrieval (k1) and LLM reranking (k2).

The overall k1-to-k2 pattern is important. Reranking reduces Keyword Stuffing most clearly, but the same reranker layer does not substantially reduce Authority Spoof or Semantic Mimicry. Therefore, the reranker improves quality against obvious surface manipulation but should not be treated as a general poisoning defence.

5.2 Complete k2 ASR Matrix

Attack Type	Claude	Gemini	GPT-4o-mini	Llama	Average
Authority Spoof	97%	97%	97%	97%	97%
Semantic Mimicry	93%	97%	93%	97%	95%
Structured Format	83%	80%	83%	77%	81%
Contradiction	77%	87%	83%	90%	84%
Keyword Stuffing	60%	57%	60%	60%	59%

Table 6. Attack Success Rate at k2 by attack type and LLM reranker.

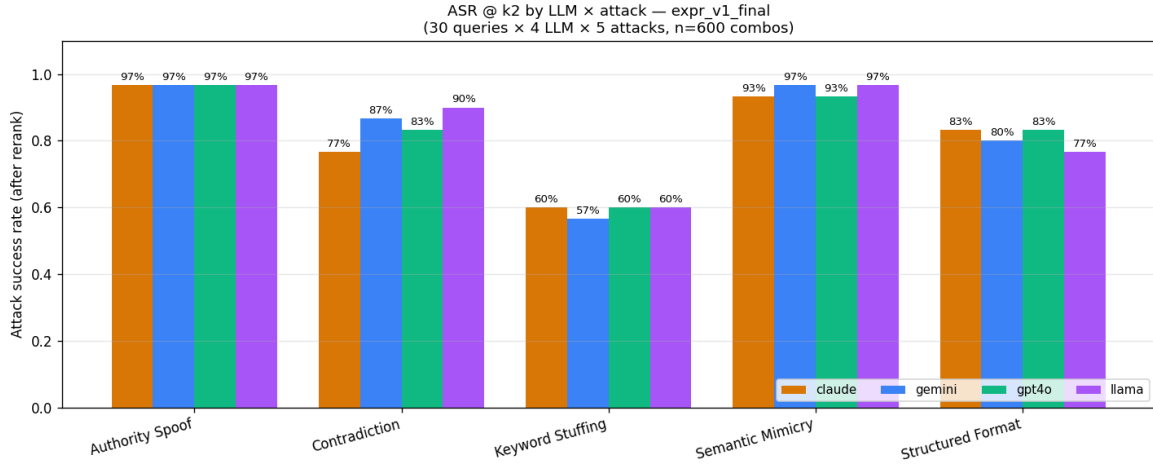


Figure 3. ASR matrix across attack types and reranker LLMs at k2.

Table 6 and Figure 3 support two broad observations. First, high-level semantic attacks remain effective across rerankers. Second, the largest model difference occurs for Contradiction rather than Authority Spoof. Authority Spoof is uniformly strong, while Contradiction exposes differences in how rerankers respond to conflicts with clean-corpus facts.

5.3 F1: Authority Spoof Is a Universal Vulnerability

Authority Spoof reaches 97% k2 ASR for all four reranker LLMs, with zero inter-model variation. This is the strongest finding in the experiment. The strategy works by embedding fabricated authority signals such as non-verifiable institutional names, DOI-like references, rankings, or expert attribution within otherwise fluent and query-relevant prose.

The result is concerning because authority-style poison is not visually suspicious in the same way as crude keyword stuffing. A downstream system or evaluator may treat such text as high-quality precisely because it appears formal and well-supported. The finding suggests that LLM rerankers can confuse authority-style presentation with relevance or trustworthiness when explicit source verification is absent.

5.4 F2: Contradiction Shows the Largest Cross-Model Robustness Gap

Reranker LLM	Contradiction k2 ASR	Status
Claude Sonnet 4.5	77%	Most robust
GPT-4o-mini	83%	
Gemini 2.5 Flash Lite	87%	
Llama 3.3 70B	90%	Most vulnerable

Table 7. Contradiction attack ASR at k2 by LLM reranker.

The gap between Claude and Llama is 13 percentage points, the largest cross-model difference observed in the experiment. This indicates that reranker model choice affects poisoning robustness, especially when the poison conflicts with existing corpus facts. However, the result should not be misread as showing that Claude solves the problem.

Even the most robust model in this group still allows Contradiction poison documents into top-k2 for 77% of trials.

5.5 F3: Keyword Stuffing Is the Only Attack Clearly Weakened by Reranking

Stage	Keyword Stuffing ASR	Change
k1 dense retriever	73%	-
k2 LLM reranker	59%	-14 percentage points

Table 8. Keyword Stuffing ASR at k1 and k2, averaged across rerankers.

Keyword Stuffing is the only attack type that is substantially reduced between dense retrieval and LLM reranking. The 14 percentage point reduction suggests that LLM rerankers can demote documents relying on obvious lexical repetition or poor fluency. However, this improvement does not generalise to more subtle attacks. Semantic Mimicry and Authority Spoof remain highly successful after reranking, suggesting that the reranker is better at rejecting obvious spam than detecting plausible misinformation.

5.6 O1: Reranker Output Completeness Differs Across LLMs

LLM	padded > 0 / 150	Rate	padded > 5 / 150	Rate
Claude	1	0.7%	1	0.7%
Gemini 2.5 Flash Lite	3	2.0%	0	0%
GPT-4o-mini	20	13.3%	1	0.7%
Llama	4	2.7%	1	0.7%
Total	28 / 600	4.7%	3 / 600	0.5%

Table 9. Reranker output completeness telemetry.

GPT-4o-mini has the highest under-output rate, with incomplete listwise ranking output in 13.3% of rows. This is an implementation-relevant finding: LLM-as-reranker systems should monitor output completeness and expose fallback behaviour rather than silently accepting malformed rankings.

However, this does not invalidate the main results. The reported k2 metric evaluates only top-5. Parser fill affects the top-5 metric only when the LLM emits fewer than five rankings, i.e., padded > 5. This stricter condition occurs in only 3 out of 600 rows, or 0.5%. Sensitivity analysis after removing these rows changes every attack-by-LLM ASR by no more than 0.6 percentage points, and the three main findings remain unchanged.

5.7 O2: Gemini Does Not Simply Pass Through Dense Order

After excluding the few Gemini rows affected by under-output, Gemini performs slightly better than the four-model average on surface attacks such as Keyword Stuffing and Structured Format. The difference is only a few percentage points and is much smaller than the 13 percentage point Contradiction gap. Therefore, it should be treated as a secondary observation rather than a core result. Still, it suggests that Gemini is not merely passing through the dense retriever original order; it makes some independent listwise judgement.

6. Discussion

6.1 Authority Cues Are a Structural Weakness for LLM Reranking

The Authority Spoof result has the strongest deployment implication. In this experimental setting, all four rerankers assign high enough relevance to authority-style poison documents for the attack to reach 97% ASR. The likely reason is not that the models verify fake citations and accept them as true. A more cautious interpretation is that they treat markers of credibility - institutional names, study language, rankings, and DOI-like strings - as useful relevance signals when explicit verification is absent.

This distinction matters. Relevance and trustworthiness are different properties. A poison document can be highly relevant to the query while being fabricated. A safer production system should therefore avoid using LLM reranking as a trust filter. Possible defences include source verification, metadata validation, domain allowlists, citation checking, provenance scoring, or separating relevance ranking from trust assessment.

6.2 Model Choice Matters, but Is Not a Defence by Itself

The Contradiction result shows that reranker choice affects security. Claude is 13 percentage points more robust than Llama under Contradiction attacks. This may reflect differences in instruction following, factual consistency judgement, or sensitivity to conflict between candidate documents. However, the experiment does not directly identify the causal mechanism, so it would be inappropriate to claim that a specific training strategy explains the gap.

The safer conclusion is that model selection is security-relevant but incomplete. Even the strongest model in this experiment still allowed Contradiction poison documents into top-k2 in 77% of cases. Selecting a stronger reranker may reduce risk, but it cannot replace corpus governance, poisoning detection, source validation, or post-retrieval consistency checks.

6.3 Reranking Helps Mainly Against Surface-Level Manipulation

The k1-to-k2 drop for Keyword Stuffing shows that LLM reranking has value. It can reduce the effect of obvious documents that repeat keywords without providing useful content. This is consistent with the intuition that an LLM can judge fluency and usefulness better than dense similarity alone.

However, the same reranking step is much less effective against attacks that look semantically plausible. Semantic Mimicry and Authority Spoof are written to resemble legitimate documents, and Contradiction poison can directly address the query while contradicting clean facts. The reranker is optimised to rank relevance, not to perform adversarial verification. This explains why adding a reranker improves retrieval quality but does not automatically create a security boundary.

6.4 Contradiction Results Challenge Intuitive Expectations

Before the experiment, a reasonable expectation was that LLM rerankers would notice contradiction better than dense retrievers. Since Contradiction poison conflicts with existing corpus facts, the model might be expected to demote it. The actual result challenges that expectation: Contradiction still succeeds between 77% and 90% ASR depending on the reranker.

This suggests that the reranker may not be performing deep cross-document fact checking. Its prompt asks for relevance ranking, so it may prioritise documents that directly address the query even when they conflict with other candidates. Rhetorical framing may also matter: a poison document that says 'recent findings correct older information' can appear informative rather than suspicious. In practical terms, contradiction detection should be implemented explicitly if factual consistency is required; it should not be assumed as a side effect of LLM reranking.

6.5 Telemetry Is Part of RAG Security Engineering

The padded-output result is not merely an implementation detail. It shows that LLM-as-reranker components can fail in ways that are easy to miss. If an LLM emits an incomplete ranking and the parser silently fills the rest from dense order, the system may appear to have used an LLM reranker while part of the result actually comes from fallback logic. In this project, the effect on top-5 metrics is small, but in larger systems with different k values or less careful logging, such behaviour could distort both evaluation and deployment behaviour.

A secure RAG pipeline should therefore log reranker completeness, parse failures, fallback frequency, model-specific anomalies, and error conditions. These signals should be available during evaluation and monitored in production.

6.6 Implications for Production RAG Deployments

The aggregate findings imply that retrieval-stage poisoning should be treated as a first-class RAG security risk. First, systems that ingest partially trusted corpora need corpus governance before indexing. Second, ranking quality and document trustworthiness should be treated separately. Third, robustness benchmarks should be included when choosing LLM rerankers for security-sensitive use cases. Fourth, explicit checks for source validity and cross-document consistency are needed because LLM reranking alone mainly mitigates obvious surface-level manipulation.

7. Limitations

7.1 Limited Corpus Scale

The experiment uses 5,290 documents, which is much smaller than many production RAG deployments. The MS MARCO background passages make the task more realistic than a small toy corpus, but the results should not be directly generalised to million-document systems without further validation. The reported ASR values characterise this controlled benchmark rather than every possible RAG deployment.

7.2 Single Fixed Generator

The generator is fixed to Claude and is not part of the experimental design. This was an intentional scope decision to avoid a reranker-by-generator combinatorial explosion. As a result, the findings should be interpreted as retrieval- and reranking-stage results, not as claims about generator-stage answer manipulation. A poison document entering top-k2 is a necessary influence channel, but it does not prove that the final generated answer will be false.

7.3 GPT-4o-mini Output Completeness and Sensitivity Analysis

GPT-4o-mini produced incomplete listwise rankings in 20 of 150 trials, a 13.3% under-output rate. The parser backfilled missing positions using dense-retrieval order. This creates a potential measurement artefact, so the project records padding telemetry and performs a sensitivity analysis.

The important distinction is between padded > 0 and padded > 5. padded > 0 means the LLM emitted fewer than 10 rankings, but this does not necessarily affect the reported top-5 metric. padded > 5 means the LLM emitted fewer than 5 rankings, so parser fill enters the evaluated top-5 range. Only 3 out of 600 rows, or 0.5%, satisfy this stricter condition. After removing these rows, every attack-by-LLM ASR changes by no more than 0.6 percentage points. F1, F2, and F3 remain unchanged.

7.4 OpenRouter Capacity Variability

All LLM API calls are routed through OpenRouter, a third-party unified LLM gateway. Capacity and API behaviour may fluctuate, including rate-limit or timeout errors such as 429 or 504. The experiment uses retry-with-backoff, but external API instability remains a possible source of noise. The released CSV includes error and padding columns so affected rows can be audited.

7.5 Research Demonstration Scope

The system is a research demo rather than a production-grade RAG platform. It does not implement authentication, enterprise access control, persistent user data, full production monitoring, citation verification, or user-uploaded corpora. These omissions are acceptable for the research question but should be addressed before deployment in a real application.

8. Conclusion and Future Work

This report evaluates whether small-scale document poisoning can alter retrieval and reranking results in a RAG pipeline. The evaluation defines attack success as rank change rather than final-answer manipulation, making the analysis focused on the upstream context-selection stage. Across 600 trials, the results show that Authority Spoof is a universal vulnerability, reaching 97% ASR across all four LLM rerankers. Contradiction attacks reveal the largest model-level robustness gap, with Claude at 77% ASR and Llama at 90% ASR. Keyword Stuffing is the only attack clearly weakened by reranking, dropping from 73% ASR at k1 to 59% at k2.

The main conclusion is that LLM-based reranking improves resistance to simple surface-level manipulation but does not provide a reliable defence against semantically plausible or authority-style poisoning. Secure RAG deployment should therefore treat retrieval-stage poisoning as a first-class threat. Practical defences should include corpus governance, source verification, poison detection, explicit contradiction checking, and reranker telemetry rather than relying only on a stronger LLM reranker.

Future work should evaluate larger corpora, additional reranker architectures, and explicit provenance or contradiction-checking defences.

References

- [1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [2] A. Shafran, R. Schuster, and V. Shmatikov, "Machine against the RAG: Jamming retrieval-augmented generation with blocker documents," in Proc. USENIX Security Symposium, 2025. [Online]. Available: <https://www.usenix.org/system/files/usenixsecurity25-shafran.pdf>
- [3] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in Proc. USENIX Security Symposium, 2025. [Online]. Available: <https://arxiv.org/abs/2402.06363>
- [4] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in Proc. USENIX Security Symposium, 2025. [Online]. Available: <https://arxiv.org/abs/2402.07867>
- [5] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, 2019. [Online]. Available: <https://arxiv.org/abs/1702.08734>
- [6] P. Bajaj et al., "MS MARCO: A human-generated machine reading comprehension dataset," 2016. [Online]. Available: <https://arxiv.org/abs/1611.09268>
- [7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019. [Online]. Available: <https://arxiv.org/abs/1908.10084>
- [8] Sentence Transformers, "sentence-transformers/all-MiniLM-L6-v2," Hugging Face. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [9] Anthropic, "Claude Sonnet 4.5 system card," 2025. [Online]. Available: <https://www.anthropic.com/claude-sonnet-4-5-system-card>
- [10] OpenAI, "GPT-4o mini Model," OpenAI API Documentation. [Online]. Available: <https://developers.openai.com/api/docs/models/gpt-4o-mini>
- [11] Google, "Gemini 2.5 Flash-Lite," Gemini API Documentation, Google AI for Developers. [Online]. Available: <https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash-lite>
- [12] Meta, "Llama 3.3 Model Card," 2024. [Online]. Available: https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_3/
- [13] OpenRouter, "Models," OpenRouter Documentation. [Online]. Available: <https://openrouter.ai/docs/guides/overview/models>